

Федеральное государственное автономное образовательное
учреждение высшего образования «Санкт-Петербургский
национальный исследовательский Университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №6
По вычислительной математике
Вариант №2

**«Численное решение обыкновенных
дифференциальных уравнений»**

Выполнил: Гладышев Иван Станиславович
Группа: Р3213

Принимающий работу: Бострикова Дарья Константиновна

Санкт-Петербург 2026

Содержание

1	Цель лабораторной работы	2
2	Порядок выполнения работы	3
3	Рабочие формулы численных методов	4
3.1	Постановка задачи	4
3.2	Усовершенствованный метод Эйлера (2-й порядок точности) . .	4
3.3	Метод Рунге–Кутты 4-го порядка	4
3.4	Метод Милна (предиктор-корректор, 4-й порядок)	4
3.5	Оценка погрешности	5
3.6	Усовершенствованный метод Эйлера	5
3.7	Метод Рунге–Кутты 4-го порядка	6
3.8	Метод Милна	7
4	Листинг программы	8
5	Результаты выполнения программы	12
5.1	Тест 1	12
5.2	Тест 2	12
5.3	Тест 3	13
6	Выводы	14

1 Цель лабораторной работы

Решить задачу Коши для обыкновенного дифференциального уравнения первого порядка вида

$$y' = f(x, y), \quad y(x_0) = y_0$$

численными методами:

- усовершенствованный метод Эйлера (метод Эйлера с пересчетом);
- метод Рунге–Кутты 4-го порядка;
- метод Милна (многошаговый метод предиктор-корректор).

Дополнительно реализовать:

- оценку точности одношаговых методов по правилу Рунге;
- визуализацию точного и приближённых решений.

2 Порядок выполнения работы

1. Реализовать в программе три численных метода.
2. Предусмотреть ввод исходных данных с клавиатуры:
 - выбор ОДУ из нескольких предложенных;
 - начальные условия $y_0 = y(x_0)$;
 - интервал дифференцирования $[x_0, x_n]$;
 - шаг h ;
 - точность ε .
3. Вычислить приближённые значения в узлах сетки.
4. Оценить погрешность одношаговых методов по правилу Рунге.
5. Для метода Милна сравнить с точным решением.
6. Построить графики точного и приближённых решений.
7. Протестировать программу на различных наборах данных.
8. Проанализировать полученные результаты.

3 Рабочие формулы численных методов

3.1 Постановка задачи

Решается задача Коши:

$$y' = f(x, y), \quad y(x_0) = y_0, \quad x \in [x_0, x_n].$$

Вводится равномерная сетка:

$$x_i = x_0 + i \cdot h, \quad i = 0, 1, \dots, n, \quad h = \frac{x_n - x_0}{n}.$$

3.2 Усовершенствованный метод Эйлера (2-й порядок точности)

$$\tilde{y}_{i+1} = y_i + h \cdot f(x_i, y_i) \quad (\text{предиктор})$$

$$y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1})] \quad (\text{корректор})$$

3.3 Метод Рунге–Кутты 4-го порядка

$$k_1 = h \cdot f(x_i, y_i)$$

$$k_2 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right)$$

$$k_3 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right)$$

$$k_4 = h \cdot f(x_i + h, y_i + k_3)$$

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

3.4 Метод Милна (предиктор-корректор, 4-й порядок)

Предиктор

$$y_{i+1}^{(\text{прогн})} = y_{i-3} + \frac{4h}{3}(2f_{i-2} - f_{i-1} + 2f_i)$$

Корректор

$$y_{i+1}^{(\text{корр})} = y_{i-1} + \frac{h}{3}(f_{i-1} + 4f_i + f_{i+1}^{(\text{прогн})})$$

где $f_i = f(x_i, y_i)$, а $f_{i+1}^{(\text{прогн})} = f(x_{i+1}, y_{i+1}^{(\text{прогн})})$.

Для запуска метода Милна необходимо знать y_1, y_2, y_3 (получаются методом Рунге–Кутты 4-го порядка).

3.5 Оценка погрешности

Правило Рунге (для одношаговых методов):

$$R = \frac{y^h - y^{h/2}}{2^p - 1},$$

где y^h — решение с шагом h , $y^{h/2}$ — с шагом $h/2$, p — порядок точности метода. Если $|R| \leq \varepsilon$, точность считается достигнутой.

Для метода Милна:

$$\varepsilon = \max_{0 \leq i \leq n} |y_i^{\text{ТОЧН}} - y_i|.$$

3.6 Усовершенствованный метод Эйлера

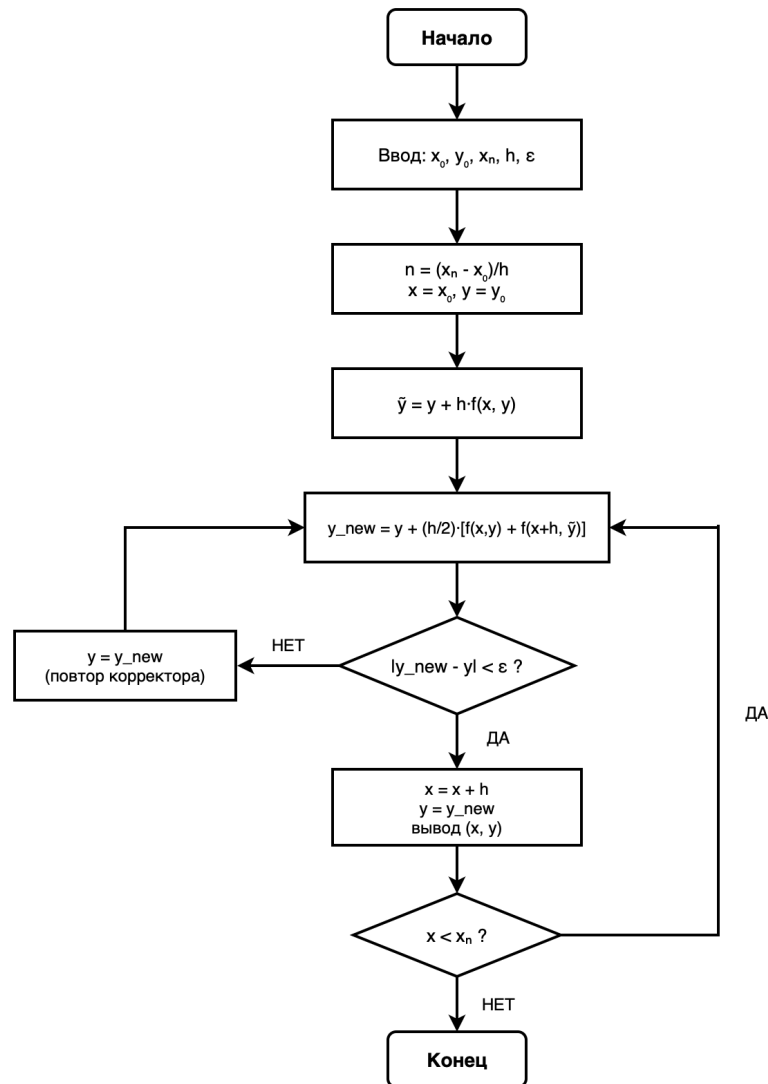


Рис. 1: Блок-схема усовершенствованного метода Эйлера

3.7 Метод Рунге–Кутты 4-го порядка

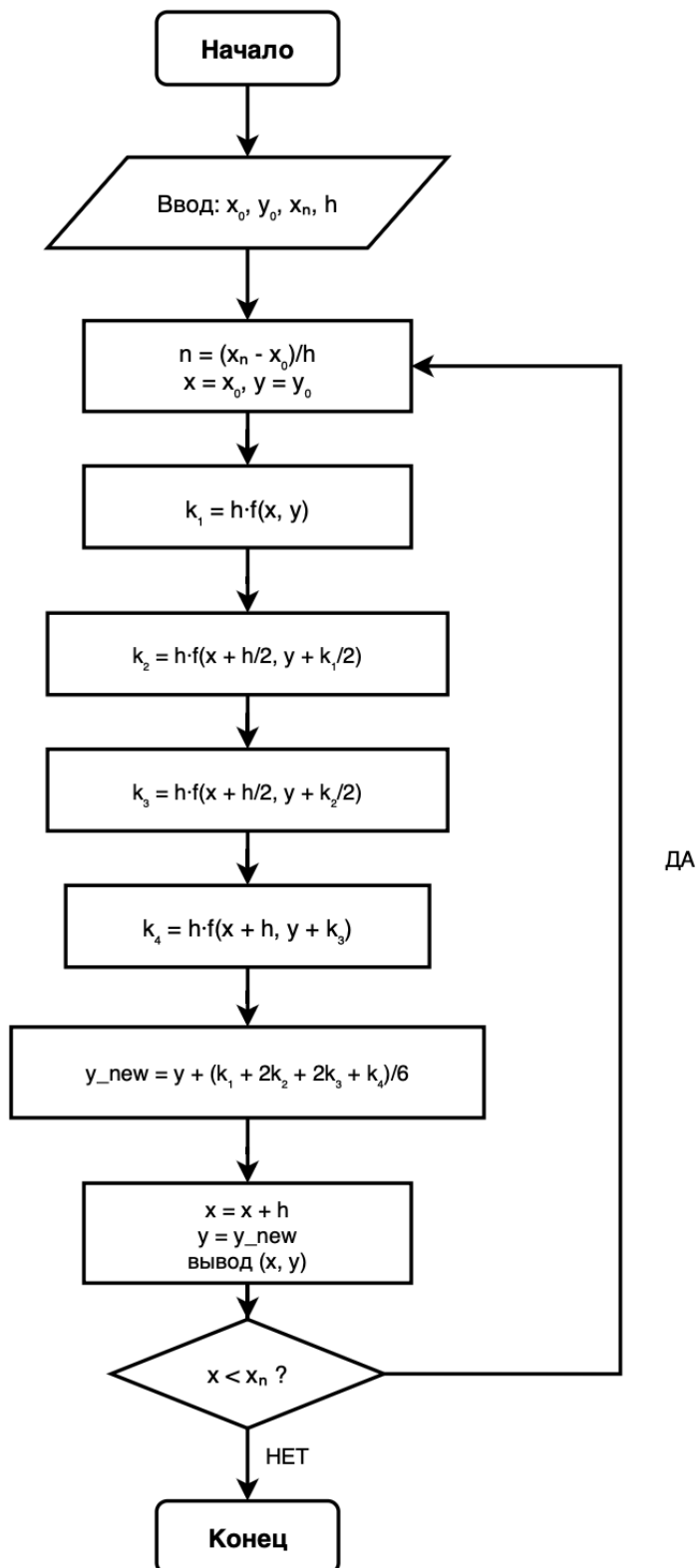


Рис. 2: Блок-схема метода Рунге–Кутты 4-го порядка

3.8 Метод Милна

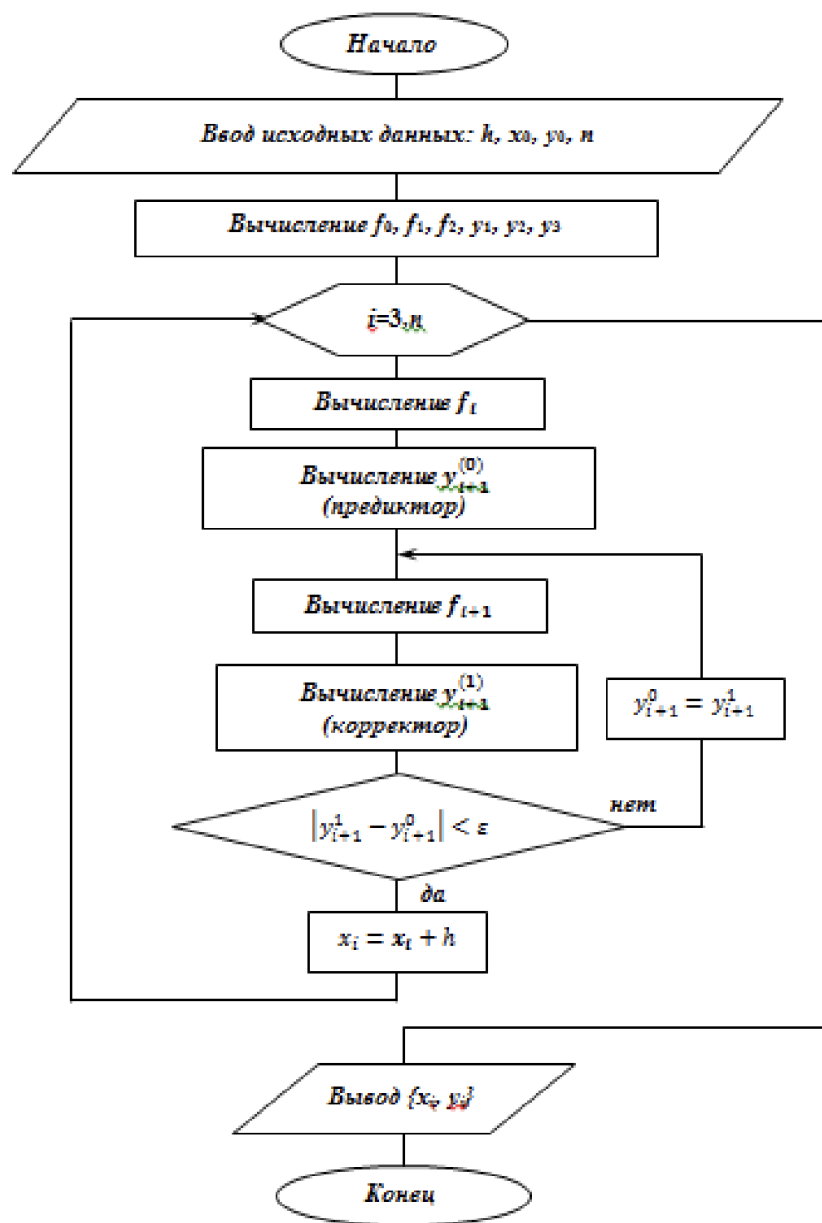


Рис. 3: Блок-схема метода Милна

4 Листинг программы

Программа реализована на языке Java. Численные методы вынесены в отдельный класс DifferentialEquationSolver.

```
1 package ru.itmo.lab6.solver;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import ru.itmo.lab6.model.DifferentialEquation;
6 import ru.itmo.lab6.model.InputData;
7 import ru.itmo.lab6.model.MethodResult;
8 import ru.itmo.lab6.model.Solutions;
9
10 public class DifferentialEquationSolver {
11     public Solutions solve(InputData input) {
12         validate(input);
13
14         List<Double> xValues = buildXValues(input.x0(), input.xn(), input.h
15         ());
16         List<Double> exactValues = calculateExactValues(input, xValues);
17
18         List<Double> improvedEuler = solveImprovedEuler(input.equation(),
19         input.x0(), input.y0(), input.h(), xValues.size());
20         double improvedEulerError = estimateRungeError(input.equation(),
21         input.x0(), input.y0(), input.xn(), input.h(), 2);
22
23         List<Double> rungeKutta = solveRungeKutta4(input.equation(), input.
24         x0(), input.y0(), input.h(), xValues.size());
25         double rungeKuttaError = estimateRungeError(input.equation(), input.
26         x0(), input.y0(), input.xn(), input.h(), 4);
27
28         List<Double> milne = solveMilne(input.equation(), input.x0(), input.
29         y0(), input.h(), xValues.size(), input.epsilon());
30         double milneError = maxAbsoluteError(milne, exactValues);
31
32         return new Solutions(
33             xValues,
34             exactValues,
35             new MethodResult("
36             ", improvedEuler, improvedEulerError, "
37             "),
38             new MethodResult("
39             -
40             4
41             ",
42             rungeKutta, rungeKuttaError, "
43             "),
44             new MethodResult("
45             ", milne, milneError, "
46             ")
47         );
48     }
49
50     private void validate(InputData input) {
51         if (input.equation() == null) {
52             throw new IllegalArgumentException("
53             .");
54         }
55         if (input.h() <= 0) {
56             throw new IllegalArgumentException("
57             h
58             .");
59         }
60     }
61 }
```

```

42     if (input.epsilon() <= 0) {
43         throw new IllegalArgumentException("epsilon
44     }
45     if (input.xn() <= input.x0()) {
46         throw new IllegalArgumentException("x0.");
47     }
48     double steps = (input.xn() - input.x0()) / input.h();
49     double rounded = Math rint(steps);
50     if (Math.abs(steps - rounded) > 1e-9) {
51         throw new IllegalArgumentException("h
52         [x0, xn]
53     }
54     if (rounded < 4) {
55         throw new IllegalArgumentException("4.");
56     }
57 );
58 }
59
60 private List<Double> buildXValues(double x0, double xn, double h) {
61     int count = (int) Math.round((xn - x0) / h) + 1;
62     List<Double> xValues = new ArrayList<>(count);
63     for (int i = 0; i < count; i++) {
64         xValues.add(x0 + i * h);
65     }
66     return xValues;
67 }
68
69 private List<Double> calculateExactValues(InputData input, List<Double>
70 xValues) {
71     List<Double> exactValues = new ArrayList<>(xValues.size());
72     for (double x : xValues) {
73         exactValues.add(input.equation().exactValue(input.x0(), input.y0
74         (), x));
75     }
76     return exactValues;
77 }
78
79 private List<Double> solveImprovedEuler(DifferentialEquation equation,
80 double x0, double y0, double h, int count) {
81     List<Double> yValues = new ArrayList<>(count);
82     yValues.add(y0);
83     double x = x0;
84     double y = y0;
85     for (int i = 1; i < count; i++) {
86         double predictor = y + h * equation.derivative(x, y);
87         y = y + h / 2.0 * (equation.derivative(x, y) + equation.
88         derivative(x + h, predictor));
89         x += h;
90         yValues.add(y);
91     }
92     return yValues;
93 }
94
95 private List<Double> solveRungeKutta4(DifferentialEquation equation,
96 double x0, double y0, double h, int count) {
97     List<Double> yValues = new ArrayList<>(count);
98     yValues.add(y0);

```

```

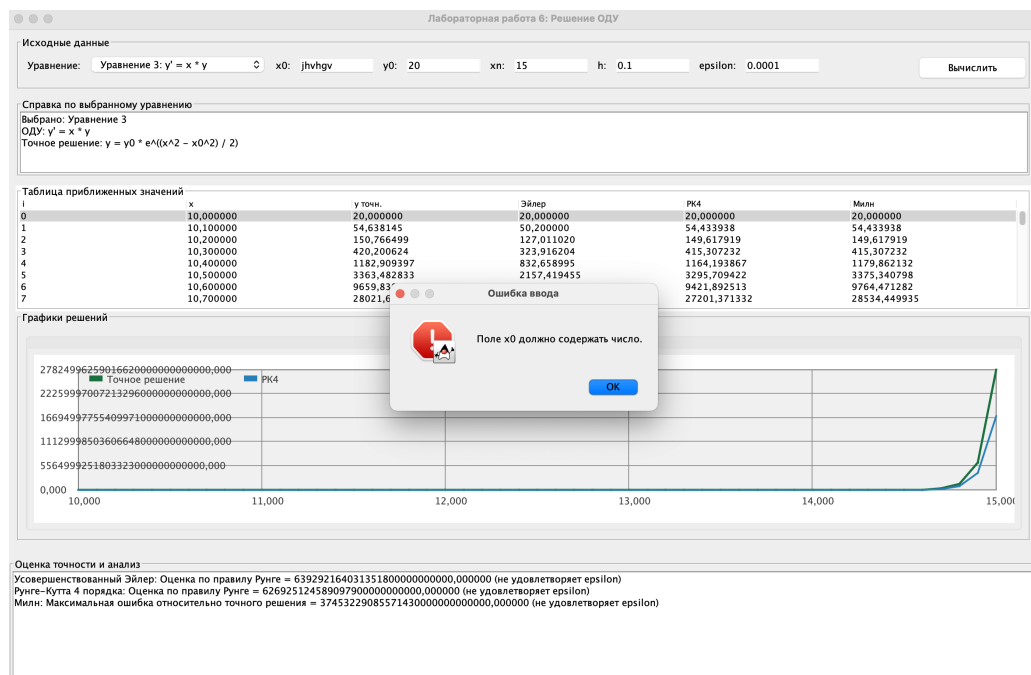
92     double x = x0;
93     double y = y0;
94     for (int i = 1; i < count; i++) {
95         double k1 = h * equation.derivative(x, y);
96         double k2 = h * equation.derivative(x + h / 2.0, y + k1 / 2.0);
97         double k3 = h * equation.derivative(x + h / 2.0, y + k2 / 2.0);
98         double k4 = h * equation.derivative(x + h, y + k3);
99         y = y + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0;
100        x += h;
101        yValues.add(y);
102    }
103    return yValues;
104 }
105
106 private List<Double> solveMilne(DifferentialEquation equation, double x0
, double y0, double h, int count, double epsilon) {
107     List<Double> yValues = new ArrayList<>(solveRungeKutta4(equation, x0
, y0, h, Math.min(count, 4)));
108     List<Double> fValues = new ArrayList<>(count);
109     for (int i = 0; i < yValues.size(); i++) {
110         double x = x0 + i * h;
111         fValues.add(equation.derivative(x, yValues.get(i)));
112     }
113     for (int i = 3; i < count - 1; i++) {
114         double xNext = x0 + (i + 1) * h;
115         double predictor = yValues.get(i - 3) + 4.0 * h / 3.0 * (2.0 *
fValues.get(i) - fValues.get(i - 1) + 2.0 * fValues.get(i - 2));
116         double corrected = predictor;
117         double previous;
118         int iterations = 0;
119         do {
120             previous = corrected;
121             corrected = yValues.get(i - 1) + h / 3.0 * (fValues.get(i -
1) + 4.0 * fValues.get(i) + equation.derivative(xNext, previous));
122             iterations++;
123         } while (Math.abs(corrected - previous) > epsilon && iterations
< 100);
124         yValues.add(corrected);
125         fValues.add(equation.derivative(xNext, corrected));
126     }
127     return yValues;
128 }
129
130 private double estimateRungeError(DifferentialEquation equation, double
x0, double y0, double xn, double h, int order) {
131     List<Double> coarse;
132     List<Double> fine;
133     int coarseCount = (int) Math.round((xn - x0) / h) + 1;
134     int fineCount = (int) Math.round((xn - x0) / (h / 2.0)) + 1;
135
136     if (order == 2) {
137         coarse = solveImprovedEuler(equation, x0, y0, h, coarseCount);
138         fine = solveImprovedEuler(equation, x0, y0, h / 2.0, fineCount);
139     } else {
140         coarse = solveRungeKutta4(equation, x0, y0, h, coarseCount);
141         fine = solveRungeKutta4(equation, x0, y0, h / 2.0, fineCount);
142     }
143
144     double maxError = 0.0;
145     for (int i = 0; i < coarse.size(); i++) {

```

```
146         double diff = Math.abs(fine.get(i * 2) - coarse.get(i)) / (Math.  
pow(2, order) - 1.0);  
147         maxError = Math.max(maxError, diff);  
148     }  
149     return maxError;  
150 }  
151  
152     private double maxAbsoluteError(List<Double> actual, List<Double>  
expected) {  
153         double max = 0.0;  
154         for (int i = 0; i < actual.size(); i++) {  
155             max = Math.max(max, Math.abs(actual.get(i) - expected.get(i)));  
156         }  
157         return max;  
158     }  
159 }
```

Листинг 1: Решатель ОДУ (DifferentialEquationSolver.java)

5.3 Тест 3



6 Выводы

В ходе выполнения лабораторной работы были реализованы и исследованы три численных метода решения задачи Коши для ОДУ первого порядка:

1. Усовершенствованный метод Эйлера:

- имеет второй порядок точности $\delta_n = O(h^2)$;
- относится к одношаговым методам, прост в реализации;
- погрешность метода накапливается и требует малого шага h для достижения удовлетворительной точности.

2. Метод Рунге–Кутты 4-го порядка:

- имеет четвёртый порядок точности $\delta_n = O(h^4)$;
- требует на каждом шаге четырёхкратного вычисления правой части $f(x, y)$;
- повышенная точность окупает вычислительные затраты, позволяя вести счёт с более крупным шагом.

3. Метод Милна:

- относится к многошаговым методам прогноза и коррекции;
- имеет четвёртый порядок точности $\delta_n = O(h^4)$;
- требует вычисления первых трёх точек другими методами (например, Рунге–Куттой) — «холодный старт»;
- экономичнее метода Рунге–Кутты: на каждом шаге требуется только два вычисления $f(x, y)$.

Правило Рунге успешно применяется для контроля точности одношаговых методов. Для многошаговых методов (Милн) удобнее сравнивать с известным точным решением.